Prefix Sums — A Study Guide for the AIO

This guide takes you from the basic prefix sum all the way to 2D prefix sums, difference arrays, and prefix/suffix extrema — the toolkit that lets you answer "sum/best over a range" questions almost instantly. All code is C++ (only `<iostream>`, 1-indexed, no template containers or newer language features), so it drops straight into a contest.

---

1. The core idea: precomputation

A very common trick in algorithm design is **precomputation**: take work that would be slow if you repeated it, and do some of it once, up front, so everything after is fast.

Many problems ask the same kind of question about one array over and over: "*what is the sum of the interval* `a[l .. r]`*?*" If you add it up from scratch every time, that's slow. Instead, spend `O(N)` once to build a **prefix sum table**, and then answer each query in `O(1)`. Trading a little setup for fast queries is one of the most important optimisations you'll use.

---

2. A motivating problem: Mansion

Problem (paraphrased): given an array of `N` integers, find a contiguous section (an **interval / subarray**) of exactly `K` elements whose sum is as large as possible. Output that sum. (Mansion, AIO 2007)

2.1 Brute force

There are exactly `N - K + 1` windows (**watch the off-by-one**). Add up the `K` values in each window and keep the best:

```
long long best = -1;
for (int start = 1; start + K - 1 <= N; start++) {
    long long cur = 0;
    for (int i = 0; i < K; i++) cur += a[start + i];   // inner loop: O(K)
    if (cur > best) best = cur;
}
```

The outer loop is `O(N)`, the inner loop is `O(K)`, so the total time is **`O(N·K)`**. On Mansion this only scores **50 / 100** — the waste is that the inner loop re-adds the whole window from scratch every time. Can we do better?

2.2 Fixing it with prefix sums

Yes: build a prefix sum table in `O(N)`, then get **each window's sum with a single subtraction in `O(1)`**. The total drops to **`O(N)`**, which scores **100 / 100**. The formula and code are in §3, and we return to Mansion in §3.5.

> There is also a **sliding window** solution that reaches `O(N)`: as the window moves right, subtract the element that leaves and add the one that enters. Both approaches are full marks, and it's worth understanding how they mirror each other.

---

3. 1D prefix sums

3.1 Definition

Let the array be `a[1], a[2], ..., a[N]` (**use 1-indexing — start indices at 1**). The prefix sum table (also called cumulative sums / partial sums) is:

```
s[0] = 0
s[i] = a[1] + a[2] + ... + a[i]
```

which you build with the recurrence:

```
s[i] = s[i-1] + a[i]
```

Read `s[i]` as "the running total from the start up to element `i`".

3.2 The key: any interval sum = a difference of two prefixes

The sum of any interval `[l, r]` is:

```
sum(l, r) = a[l] + a[l+1] + ... + a[r] = s[r] - s[l-1]
```

Picture the interval you want as the **blue** band. It equals the **red** band (the prefix all the way up to `r`) **minus** the **green** band (the prefix up to `l-1`). Both red and green are *prefixes*, so both are already stored:

```
s[r]   = a1 a2 ... a(l-1) | a(l) ... a(r)    <- red   (a prefix)
s[l-1] = a1 a2 ... a(l-1)                     <- green (a prefix)
blue   =                   a(l) ... a(r)      <- what's left after subtracting
```

Key insight: any interval sum can be written as the difference of two prefix sums. Build all the prefixes once (`O(N)`) and you can query *any* interval — not just a prefix — in `O(1)`.

3.3 1-indexing and the sentinel

When `l = 1`, you need `s[l-1] = s[0]`. Setting `s[0] = 0` (a **sentinel**) makes that case work with no special handling. Removing edge cases like this is the main reason to prefer 1-indexing — it kills a lot of off-by-one bugs.

3.4 Template (many interval queries)

```
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long a[MAXN];   // the array
long long s[MAXN];   // prefix sum table

int main() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    // build once: O(n)
    s[0] = 0;
    for (int i = 1; i <= n; i++) {
        s[i] = s[i - 1] + a[i];
    }

    // answer each query: O(1)
    while (q--) {
        int l, r;
        cin >> l >> r;
        long long ans = s[r] - s[l - 1];
        cout << ans << "\n";
    }

    return 0;
}
```

Check by hand: for `a = [3, 1, 4, 1, 5]`, `s = [0, 3, 4, 8, 9, 14]`. Then `sum(2, 4) = s[4] - s[1] = 9 - 3 = 6`, and indeed `1 + 4 + 1 = 6`.

3.5 Back to Mansion: turning 50 into 100

A window of length `K` ending at index `i` (for `i` from `K` to `N`) covers `a[i-K+1 .. i]`, so its sum is `s[i] - s[i-K]`. Scan the right endpoint and keep the best — total `O(N)`:

```
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long a[MAXN];
long long s[MAXN];

int main() {
    int n, k;
    cin >> n >> k;
    for (int i = 1; i <= n; i++) cin >> a[i];

    s[0] = 0;
    for (int i = 1; i <= n; i++) s[i] = s[i - 1] + a[i];

    long long best = -1;
    for (int i = k; i <= n; i++) {               // i is the window's right endpoint
        long long windowSum = s[i] - s[i - k];   // sum of a[i-k+1 .. i]
        if (best == -1 || windowSum > best) best = windowSum;
    }

    cout << best << "\n";
    return 0;
}
```

Compare: brute force is `O(N·K)` (50 points), prefix sums are `O(N)` (100 points). The whole difference is replacing "add up each window" with "one subtraction from a table".

A prefix sum is essentially a tiny data structure: `O(N)` to build, `O(1)` to query any interval sum — and it becomes a building block inside many bigger algorithms.

---

4. Difference arrays: the inverse of a prefix sum

Prefix sums handle "the array is fixed, query interval sums repeatedly." The **difference array** is the inverse and handles the opposite: **apply many range updates, then read the array once at the end.**

4.1 The problem

> Start with an array of all zeros. Apply `M` operations; each adds `v` to every element of an interval `[l, r]`. After all operations, output the final array.

Doing each update by looping is `O(N)` per update, `O(N·M)` total — too slow (TLE).

4.2 The method

For each "add `v` to `[l, r]`", change just two positions:

```
diff[l]   += v;
diff[r+1] -= v;
```

After all the updates, **take one prefix sum of `diff` to rebuild the final array**:

```
a[i] = diff[1] + diff[2] + ... + diff[i]
```

Why it works: `diff[l] += v` says "the `+v` effect starts at `l`", and `diff[r+1] -= v` says "that effect stops after `r`". Prefix-summing `diff` accumulates every still-active effect at each position.

> One line to remember: **a prefix sum turns per-element values into a running total; a difference array does the reverse, turning a running total back into per-element changes. They are inverse operations.**

4.3 Code

```cpp
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long diff[MAXN];   // note: index r+1 must be valid, so size it n+2 or MAXN

int main() {
    int n, m;
    cin >> n >> m;

    for (int k = 0; k < m; k++) {
        int l, r;
        long long v;
        cin >> l >> r >> v;
        diff[l]     += v;
        diff[r + 1] -= v;
    }

    // prefix sum of diff = rebuild the final array
    long long cur = 0;
    for (int i = 1; i <= n; i++) {
        cur += diff[i];
        cout << cur << " ";
    }
    cout << "\n";

    return 0;
}
```

---

5. 2D prefix sums

Grid problems are common: given an `N × M` matrix, answer many queries for the sum inside a **subrectangle**. The 1D idea extends naturally to two dimensions.

5.1 Definition

For a matrix `a[i][j]` (rows and columns both starting at 1), define:

```
s[i][j] = sum of every element in the rectangle from (1,1) to (i,j)
```

The recurrence uses **inclusion–exclusion**:

```
s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j]
```

Why subtract `s[i-1][j-1]`? The block above (`s[i-1][j]`) and the block to the left (`s[i][j-1]`) both include the top-left corner `s[i-1][j-1]`, so it's **counted twice** — subtract it once to compensate.

```
    +----------------+
    |   s[i-1][j]    |   <- block above
    |  +----------+  |
    |  |s[i-1][j-1]|  |   <- covered by BOTH the block above and the block left
    +--+----------+  |
    |   s[i][j-1]    |   <- block to the left
    +----------------+
```

5.2 Querying any subrectangle

For the rectangle with top-left `(r1, c1)` and bottom-right `(r2, c2)`:

```
sum = s[r2][c2]
    - s[r1-1][c2]      <- remove the strip above
    - s[r2][c1-1]      <- remove the strip to the left
    + s[r1-1][c1-1]    <- both removals took out the same top-left block; add it back once
```

Again it's inclusion–exclusion: subtract two blocks, add back the one you removed twice.

5.3 Code

```
#include <iostream>
using namespace std;

const int MAXN = 1005;
long long a[MAXN][MAXN];
long long s[MAXN][MAXN];

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> a[i][j];

    // build the 2D prefix sums: O(n*m)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j];

    // answer each subrectangle query: O(1)
    while (q--) {
        int r1, c1, r2, c2;
        cin >> r1 >> c1 >> r2 >> c2;
        long long ans = s[r2][c2] - s[r1-1][c2] - s[r2][c1-1] + s[r1-1][c1-1];
        cout << ans << "\n";
    }

    return 0;
}
```

Because the code is 1-indexed, `s[0][...]` and `s[...][0]` are naturally 0 (global arrays start zeroed), so the boundaries take care of themselves.

---

6. The wider family: suffix sums and prefix/suffix extrema

A prefix sum is one member of a whole family of "scan once" tools. They all share the same skeleton: **scan from one end to the other, keeping the running result as you go.** Change the operation you accumulate and you get a different tool.

6.1 Suffix sums

Accumulate from right to left, so `suf[i]` is `a[i] + a[i+1] + ... + a[N]`:

```
suf[n + 1] = 0;
for (int i = n; i >= 1; i--)
    suf[i] = suf[i + 1] + a[i];
```

Use it whenever you need "the sum of everything to the right of a position".

6.2 Prefix maxima / minima

`pmax[i]` is the maximum of `a[1..i]` (using a ternary instead of `std::max` keeps it template-free and explicit):

```
for (int i = 1; i <= n; i++) {
    if (i == 1) pmax[i] = a[i];
    else        pmax[i] = (pmax[i-1] > a[i]) ? pmax[i-1] : a[i];
}
```

For prefix minima, change `>` to `<`.

6.3 Suffix maxima / minima

`smax[i]` is the maximum of `a[i..n]`, scanning right to left:

```
for (int i = n; i >= 1; i--) {
    if (i == n) smax[i] = a[i];
    else        smax[i] = (smax[i+1] > a[i]) ? smax[i+1] : a[i];
}
```

These pair naturally with prefix sums, especially in problems that ask, for each position, "what is the **best value to my left / to my right**?"

---

7. A key distinction: invertible vs not

This point is easy to miss but matters a lot when you're deciding which tool fits:

- A prefix **sum** can answer **any interval** `[l, r]` via `s[r] - s[l-1]`. That uses **subtraction**, the **inverse** of addition. Because addition is invertible, you can strip off the prefix you don't want.
- A prefix **maximum** cannot. `max` has **no inverse** — you can't remove the effect of `max(a[1..l-1])` from `max(a[1..r])`. So `pmax` only answers "from the start up to here" (a prefix range), and `smax` only answers "from here to the end" (a suffix range).

So: - Invertible operations (sum, XOR) → you can query **any** interval. - Non-invertible operations (max, min, gcd) → prefix/suffix arrays only answer ranges with one **fixed endpoint**. Arbitrary-range max/min needs a more advanced structure (a Sparse Table or Segment Tree).

If you only have prefix sums so far and a problem seems to want "max over an arbitrary range", look again — very often it only needs a prefix maximum or a suffix maximum, with the endpoint pinned to one side, and a single scan is enough.

---

8. Worked examples

Example A: the most balanced split (prefix sum + total)

> Split the array into a left and a right part so that the **absolute difference between the two sides' sums is as small as possible**. Output that minimum difference.

The left sum is `s[i]`; the right sum is `total - s[i]`. Scan the split point:

```
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long a[MAXN];
long long s[MAXN];

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= n; i++) s[i] = s[i-1] + a[i];

    long long total = s[n];
    long long best = -1;
    int bestPos = 0;

    for (int i = 1; i < n; i++) {          // split between element i and i+1
        long long leftSum  = s[i];
        long long rightSum = total - s[i];
        long long d = leftSum - rightSum;
        if (d < 0) d = -d;                 // absolute value
        if (best == -1 || d < best) {
            best = d;
            bestPos = i;
        }
    }

    cout << "split after index " << bestPos << ", min difference = " << best << "\n";
    return 0;
}
```

Example B: trapping rain water (prefix maxima + suffix maxima together)

> Given bar heights `h[1..N]`, the water held above position `i` is `min(tallest bar to its left, tallest bar to its right) - h[i]`. Find the total water.

This is the classic pairing: "best on the left = prefix maxima, best on the right = suffix maxima":

```cpp
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long h[MAXN];
long long pmax[MAXN];   // prefix maxima
long long smax[MAXN];   // suffix maxima

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> h[i];

    for (int i = 1; i <= n; i++)
        pmax[i] = (i == 1) ? h[i] : (pmax[i-1] > h[i] ? pmax[i-1] : h[i]);

    for (int i = n; i >= 1; i--)
        smax[i] = (i == n) ? h[i] : (smax[i+1] > h[i] ? smax[i+1] : h[i]);

    long long total = 0;
    for (int i = 1; i <= n; i++) {
        long long left  = pmax[i];
        long long right = smax[i];
        long long lower = (left < right) ? left : right;   // the shorter side sets the leve
        total += lower - h[i];    // pmax[i] already includes h[i], so this is never negativ
    }

    cout << total << "\n";
    return 0;
}
```

Example C: subrectangle statistics (2D prefix sum)

This is exactly the §5 template: a grid, with many queries for a subrectangle's sum. A common variant makes the grid 0/1 and asks how many 1s lie inside a rectangle — same 2D prefix sum, same `O(1)` query.

> **Extension (prefix product × suffix product):** to compute, for each position, the product of all the *other* elements without using division, use a prefix product and a suffix product: `ans[i] = prefixProduct[i-1] × suffixProduct[i+1]`. It's the same "stitch the left and right information together" idea as prefix maxima × suffix maxima.

---

9. Common pitfalls

1. **Overflow.** Prefix sums grow quickly and overflow `int` (past ~2.1 billion). **Always use `long long`** for prefix sums.
2. **Array bounds.** A 1D difference array touches `diff[r+1]`, so size it `n+2`; a 2D prefix sum reads `s[0][j]` and `s[i][0]`, so row 0 and column 0 must exist.
3. **Initialise `s[0] = 0`.** Global arrays start zeroed, but clear them yourself for local arrays or when a problem has multiple test cases.

4. **Stay 1-indexed.** Keep `s[0] = 0` as a sentinel to remove most boundary cases, and don't mix 0-indexed and 1-indexed in the same solution.
5. **Query vs update.** Fixed array, repeated interval-sum queries → prefix sum. Repeated range adds, read the result at the end → difference array.
6. **Don't force a non-invertible op into an arbitrary-range query.** Arbitrary-range max/min needs a Sparse Table or Segment Tree.

---

10. Complexity cheat sheet

Technique	Build	Query / update	Use for
1D prefix sum	`O(n)`	`O(1)` query	sum of any interval
Difference array	`O(1)` per update	`O(n)` to rebuild	many range-add updates
2D prefix sum	`O(n·m)`	`O(1)` query	subrectangle sums
Prefix / suffix extrema	`O(n)`	`O(1)` query	best value with a fixed endpoint

A naive interval sum is `O(n)` per query, so `O(n·q)` overall; prefix sums cut that to `O(n + q)`, which is often the line between a solution that passes and one that times out.

---

11. Practice problems

Try each yourself for at least 15 minutes before looking at an editorial.

1. **Mansion (AIO 2007)** — a direct prefix-sum warm-up; §2 and §3 apply as-is. https://orac.amt.edu.au/problem/aio07mansionsen/
2. **Alien (AIO 2011)** — a step up. Hint: finding the best "launch position" is a lot like finding the best interval in Mansion. Think about how to **transform the input first**, then reuse the Mansion idea. https://orac.amt.edu.au/problem/aio11aliens/
3. **Two Pointers / sliding window** — an alternative `O(N)` route for Mansion that reinforces the same idea. https://orac.amt.edu.au/hub/train/aio-twopointers
4. **Backpacking (AIO 2024)** — shows a prefix sum used as one piece of a larger algorithm. https://orac.amt.edu.au/hub/train/editorials-aio24-q4

How to practise well: write out the 1D prefix sum, 2D prefix sum, and difference array templates from memory and check them by hand. Then train yourself to recognise the signal in a problem: "many interval sums" → prefix sum; "subrectangle statistics on a grid" → 2D prefix sum; "many range adds, output at the end" → difference array; "the best value to each position's left or right" → prefix / suffix extrema.

More problems: https://orac.amt.edu.au/hub/train/aio

Once you own this set of "scan once and reuse" techniques — prefix sums, difference arrays, 2D prefix sums, and prefix/suffix extrema — a large class of range problems becomes routine. And the underlying idea (compute repeated work once, store it, and reuse it) keeps coming back in more advanced topics later, so time spent here pays off.